

Summe berechnen – Verbrauchswerte aufsummieren (typisch 2019–2025)

Pseudocode (IHK-Stil):

```
Funktion summe(werte: Array<Integer>) → Integer
    s = 0
    for i = 0 to länge(werte) - 1
        s = s + werte[i]
    end for
    return s
end Funktion
```

Java-Beispiel:

```
public static int summe(int[] werte) {
    int s = 0;
    for (int i = 0; i < werte.length; i++) {
        s += werte[i];
    }
    return s;
}
```

Erklärung & Beispiel:

Ablauf: Einfache Akkumulation über alle Elemente.

Beispiel: werte = [5, 7, 9] → s=0→5→12→21 → Ergebnis 21.

Komplexität: $O(n)$.

Typische Fehler: s nicht initialisieren; falscher Datentyp (int vs. long bei großen Summen).

Prüfungstipp: Bei Geldbeträgen in Java BigDecimal oder long (Cent) erwähnen.

Durchschnitt (arithmetisches Mittel) – Prüfungsklassiker

Pseudocode (IHK-Stil):

```
Funktion durchschnitt(werte: Array<Integer>) → Real
    if länge(werte) == 0 dann
        return 0.0
    end if
    s = 0
    for i = 0 to länge(werte) - 1
        s = s + wert[i]
    end for
    return s / länge(werte)
end Funktion
```

Java-Beispiel:

```
public static double durchschnitt(int[] wert) {
    if (wert.length == 0) return 0.0;
    long s = 0;
    for (int v : wert) s += v;
    return s / (double) wert.length;
}
```

Erklärung & Beispiel:

Ablauf: Summe durch Anzahl teilen, leeres Array absichern (sonst Division durch 0).

Beispiel: [5,7,9] → Summe=21 → Durchschnitt=21/3=7.0.

Komplexität: $O(n)$.

Typische Fehler: Integer-Division (in Java → cast auf double!).

Prüfungstipp: Bei großen Zahlen Summenvariable als long verwenden.

Min/Max bestimmen – z. B. minVerbrauch / maxVerbrauch

Pseudocode (IHK-Stil):

```
Funktion minMax(werte: Array<Integer>) → Tupel<Integer,Integer>
  if länge(werte) == 0 dann
    return (0, 0)
  end if
  minWert = MAX_INT
  maxWert = MIN_INT
  for i = 0 to länge(werte) - 1
    if werte[i] < minWert dann
      minWert = werte[i]
    end if
    if werte[i] > maxWert dann
      maxWert = werte[i]
    end if
  end for
  return (minWert, maxWert)
end Funktion
```

Java-Beispiel:

```
public static int[] minMax(int[] werte) {
  if (werte.length == 0) return new int[]{0,0};
  int min = Integer.MAX_VALUE;
  int max = Integer.MIN_VALUE;
  for (int v : werte) {
    if (v < min) min = v;
    if (v > max) max = v;
  }
  return new int[]{min, max};
}
```

Erklärung & Beispiel:

Ablauf: Durchlauf mit zwei Wächterwerten (min=MAX_VALUE, max=MIN_VALUE).

Beispiel: [8,3,12,5] → min=3, max=12.

Komplexität: O(n).

Typische Fehler: min/max falsch initialisieren; nur else-if statt zwei unabhängiger ifs.

Prüfungstipp: Initialisierung mit MAX/MIN ist Standard im Pseudocode.

Grenzwertprüfung – alle Werte > Limit sammeln (Limit-Verbraucher)

Pseudocode (IHK-Stil):

```
Funktion werteUeberLimit(werte: Array<Integer>, limit: Integer) → Liste<Integer>
    erg = neue Liste<Integer>()
    for i = 0 to länge(werte) - 1
        if werte[i] > limit dann
            erg.add(werte[i])
        end if
    end for
    return erg
end Funktion
```

Java-Beispiel:

```
public static java.util.List<Integer> werteUeberLimit(int[] werte, int limit) {
    java.util.List<Integer> erg = new java.util.ArrayList<>();
    for (int v : werte) if (v > limit) erg.add(v);
    return erg;
}
```

Erklärung & Beispiel:

Ablauf: Filtere alle Werte, die über dem Grenzwert liegen, in eine Liste.

Beispiel: werte=[5,12,7,20], limit=10 → Ergebnis=[12,20].

Komplexität: $O(n)$.

Typische Fehler: $\geq$ statt $>$ (wenn die Aufgabe explizit „über“ sagt).

Prüfungstipp: Gern wird zusätzlich die **Anzahl** verlangt → `erg.size()`.

Zählen – Anzahl der Werte $\geq$ Schwelle

Pseudocode (IHK-Stil):

```
Funktion zaehleAb(werte: Array<Integer>, schwelle: Integer) → Integer
    count = 0
    for i = 0 to länge(werte) - 1
        if werte[i]  $\geq$  schwelle dann
            count = count + 1
        end if
    end for
    return count
end Funktion
```

Java-Beispiel:

```
public static int zaehleAb(int[] werte, int schwelle) {
    int count = 0;
    for (int v : werte) if (v  $\geq$  schwelle) count++;
    return count;
}
```

Erklärung & Beispiel:

Ablauf: Zähle Elemente, die eine Bedingung erfüllen ($\geq$ schwelle).

Beispiel: [3,10,10,1], schwelle=10 $\rightarrow$ count=2.

Komplexität: $O(n)$.

Typische Fehler: falscher Vergleichsoperator ($>$ statt $\geq$).

Prüfungstipp: Zählen und Sammeln (Liste) werden oft kombiniert gefragt.

Jahresstatistik erstellen – Objekt mit min, max, limitVerbraucher (IHK-Stil)

Pseudocode (IHK-Stil):

```
// Rückgabe als Objekt vom Typ Jahresstatistik(minVerbrauch: Integer, maxVerbrauch: Integer)
Funktion erstelleJahresstatistik(werte: Array<Integer>, limit: Integer) → Jahresstatistik
    minWert = MAX_INT
    maxWert = MIN_INT
    liste = neue Liste<Integer>()

    for i = 0 to länge(werte) - 1
        w = werte[i]
        if w < minWert dann
            minWert = w
        end if
        if w > maxWert dann
            maxWert = w
        end if
        if w > limit dann
            liste.add(w)
        end if
    end for

    return neues Jahresstatistik(minWert, maxWert, liste)
end Funktion
```

Java-Beispiel:

```
// Datenträger-Klasse (Record-ähnlich)
public static class Jahresstatistik {
    public final int minVerbrauch;
    public final int maxVerbrauch;
    public final java.util.List<Integer> limitVerbraucher;
    public Jahresstatistik(int minV, int maxV, java.util.List<Integer> lv) {
        this.minVerbrauch = minV;
        this.maxVerbrauch = maxV;
        this.limitVerbraucher = lv;
    }
}

public static Jahresstatistik erstelleJahresstatistik(int[] werte, int limit) {
    int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
    java.util.List<Integer> liste = new java.util.ArrayList<>();
    for (int w : werte) {
        if (w < min) min = w;
        if (w > max) max = w;
        if (w > limit) liste.add(w);
    }
    return new Jahresstatistik(min, max, liste);
}
```

Erklärung & Beispiel:

Ablauf: In **einem** Durchlauf min/max ermitteln und alle Werte über dem Limit sammeln; anschließend Objekt bauen.

Beispiel: werte=[8,3,12,5], limit=7 → min=3, max=12, liste=[8,12].

Komplexität: $O(n)$, Speicher: $O(k)$ für die Trefferliste.

Typische Fehler: min/max falsch initialisiert; Liste vergessen; Rückgabetyt nicht eindeutig.

Prüfungstipp: „Rückgabewert ist ein Objekt ...“ → in Pseudocode klar benennen (Konstruktor-ähnlich).